

PRÁCTICA DE CAMPO 7

Análisis Big Data con Apache Spark y evaluación de escalabilidad

1. Objetivo

Evaluar el uso de Apache Spark para el procesamiento y análisis de datos, comparando su ejecución con SQL Server mediante una operación de agregación sobre la tabla FactVentas del Data Warehouse AdventureWorksDW_PC5.

El análisis busca determinar la eficiencia y escalabilidad de ambas soluciones según el volumen de datos y el tipo de procesamiento requerido.

2. Entorno de trabajo

Para el procesamiento se utilizó:

Componente	Herramienta
Lenguaje	Python
Motor Big Data	Apache Spark
API	PySpark
Interfaz	Jupyter Notebook
Base de datos	SQL Server
Conexión	JDBC
Data Warehouse	AdventureWorksDW_PC5
Tabla analizada	FactVentas

La aplicación Spark fue inicializada utilizando:

```
spark = (  
    SparkSession.builder  
        .master("local[*]")  
        .appName("Spark_AdventureWorks")  
        .config("spark.jars", JDBC_JAR)  
        .getOrCreate()  
)
```

3. Carga y procesamiento de datos

Se cargó mediante JDBC la tabla FactVentas desde AdventureWorksDW_PC5 hacia un DataFrame de Spark.

La carga permitió trabajar con 60,398 registros, manteniendo la estructura de la tabla y sus principales medidas de ventas.

Posteriormente, el DataFrame se registró como una vista temporal denominada ventas, permitiendo utilizar SparkSQL para ejecutar consultas analíticas.

De esta manera se utilizaron dos objetos/componentes requeridos por la práctica:

- DataFrame
- SparkSQL

4. Agregación de indicadores

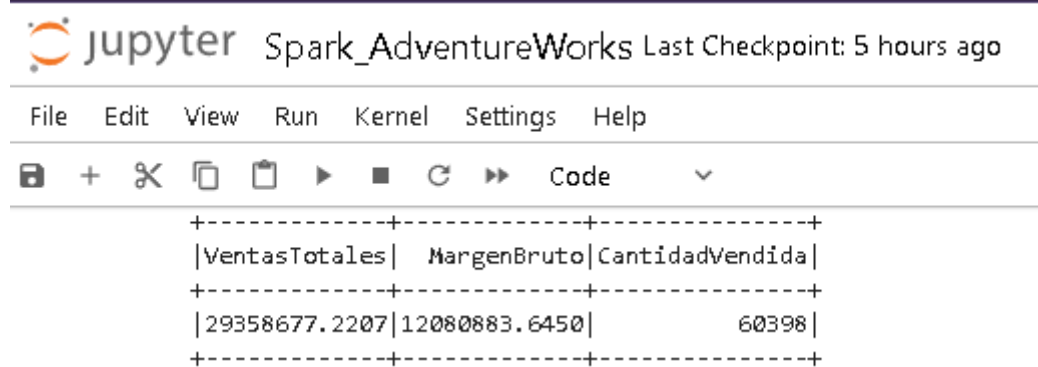
Se ejecutó una operación de agregación sobre FactVentas para obtener tres indicadores:

- Ventas Totales
- Margen Bruto

- Cantidad Vendida

La consulta utilizada fue:

```
SELECT
    SUM(ImporteVenta) AS VentasTotales,
    SUM(ImporteVenta) - SUM(CostoTotalProducto) AS MargenBruto,
    SUM(Cantidad) AS CantidadVendida
FROM ventas
```



The screenshot shows the Jupyter Spark_AdventureWorks interface. The top bar indicates the last checkpoint was 5 hours ago. The menu bar includes File, Edit, View, Run, Kernel, Settings, and Help. Below the menu is a toolbar with icons for saving, adding, deleting, copying, pasting, running, and code execution. The main area displays the query results in a table format.

VentasTotales	MargenBruto	CantidadVendida
29358677.2207	12080883.6450	60398

Los resultados coinciden con los indicadores previamente establecidos para FactVentas, donde las ventas totales corresponden a la suma de ImporteVenta y la cantidad vendida a la suma de Cantidad.

5. Comparación de tiempos

Se ejecutó la misma operación de agregación en SQL Server y Apache Spark.



The table compares the execution times of the same aggregation query in SQL Server and Apache Spark. The left column shows the results from the 'Resultados' tab in the Jupyter interface, and the right column shows the results from the 'Mensajes' tab.

SQL Server	Apache Spark
(1 fila afectada)	
SQL Server Execution Times:	
CPU time = 0 ms, elapsed time = 18 ms.	
SQL Server parse and compile time:	
CPU time = 0 ms, elapsed time = 0 ms.	
Hora de finalización: 2026-10-02T00:05:53.9740791-05:00	Tiempo Spark: 0.1716 segundos

Para facilitar la comparación:

Spark: 171.6 ms

SQL Server: 18 ms

En este escenario, SQL Server presentó un menor tiempo de ejecución.

6. Análisis de eficiencia

Para el volumen analizado de 60,398 registros, SQL Server obtuvo un mejor tiempo de respuesta que Spark.

Esto puede explicarse por las características del escenario utilizado: se trabajó con un volumen moderado de datos y Spark fue ejecutado localmente, por lo que la

inicialización y coordinación de su procesamiento representan un costo que puede ser significativo frente a una consulta de agregación relativamente sencilla.

Por tanto, para consultas analíticas sobre volúmenes moderados almacenados directamente en un Data Warehouse relacional, SQL Server puede proporcionar una respuesta más eficiente.

7. Análisis de escalabilidad

El menor tiempo obtenido por SQL Server en esta prueba no significa que Spark sea menos adecuado para Big Data.

Apache Spark está diseñado para distribuir el procesamiento entre múltiples recursos, por lo que su ventaja se vuelve más relevante cuando aumenta considerablemente el volumen de información o cuando se requiere integrar y procesar datos provenientes de diferentes fuentes.

En consecuencia:

- **SQL Server:** Resulta adecuado para consultas estructuradas, agregaciones y análisis sobre volúmenes controlados dentro del Data Warehouse.
- **Apache Spark:** Resulta conveniente cuando el volumen de datos crece significativamente y se requiere procesamiento distribuido, integración de múltiples fuentes o transformaciones analíticas de mayor complejidad.

8. Conclusiones

- Se logró procesar la tabla FactVentas mediante PySpark utilizando DataFrames y SparkSQL, cumpliendo los requisitos técnicos establecidos para el análisis Big Data.
- La operación de agregación permitió obtener 29 millones en ventas totales, 12 millones de margen bruto y 60,398 unidades vendidas.
- Los resultados obtenidos mediante Spark son consistentes con los indicadores definidos previamente para el Data Warehouse.
- En la prueba realizada, SQL Server presentó un menor tiempo de ejecución (18 ms) frente a Apache Spark (171.6 ms).
- Para el volumen moderado utilizado en esta experiencia, SQL Server resulta más eficiente para la operación evaluada.
- Apache Spark mantiene una ventaja potencial en escenarios de mayor volumen y procesamiento distribuido, por lo que su conveniencia debe evaluarse considerando el crecimiento de los datos y la complejidad de las operaciones.